

Server Side Vulnerabilities

Dr. Ramokapane

Injection Vulnerabilities

Insufficient input validation on data coming from the client

which means?

Code is inserted into the program control flow on the server-side

Circumvent authentication

Leak sensitive data or privileged information

Command Injection

Run arbitrary shell code (or other commands) on the host operating system

With the same privilege as the host application!

Lab 1

1. Start up VulnerableApp as instructed.
2. Go to the Injection Vulnerabilities.
3. Complete all six levels. Your goal is to get the system to return responses from Linux shell commands.

CWE 78

Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

Can have a number of variations

<system command>([arg])

<system command> *(as is the case in our lab example)*

CWE 78 - Mitigations

Several ways to address this

Inclusion of “allow list” - known good data formats and exclusion of all others

Let's look at the code

Parameterisation to functions where the parameter is checked before execution

Fixed input values

Sandboxing

Do not only check on the client side!

SQL Injection

SQL: Structured Query Language

```
vuln_statement = " 'SELECT _*_FROM_creditcards_WHERE_number=_ ' " +  
    user_input + "; "
```

retrieves?

```
vuln_statement = " 'SELECT _*_FROM_creditcards_WHERE_number=_ ' '  
    OR_ '1 '= '1 ' ; "
```

retrieves?

CWE 89

Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

A common issue with database-driven web sites

Depends on the fact that SQL makes no real distinction between the control and data planes

Without sufficient removal or quoting of SQL syntax in user-controllable inputs, the generated SQL query can cause those inputs to be interpreted as SQL instead of ordinary user data

This can be used to alter query logic to bypass security checks, or to insert additional statements that modify the back-end database, possibly including execution of system commands

Types of Injections

Blind: No results are displayed back to the attacker but the impact of the attack may be visible through a true/false response or what is/isn't displayed on the website as a consequence.

Error-based: Responses return SQL errors which can be informative for the attacker to understand the underlying database, e.g., its schema, to craft further injection attempts.

Second order: Attacker provided input is stored in the database for later use. Other parts of the application utilise it without escaping/filtering/validating properly.

SQL Injection

Lab 2

1. Start up Juice Shop as instructed.
2. Study the interface and features that it offers as an application. It is important to know the application before you start trying to explore its vulnerabilities.
3. As a first step find the hidden score board (hint: this is not an input validation vulnerability)
4. Once you have found the score board, login as admin.

CWE 89 - Mitigations

Similar to CWE 78

Inclusion of known good data formats and exclusion of all others

Use *prepared statements*. Placeholder (or bind) variables that only accept values of a given type and no arbitrary SQL code

Use APIs or libraries to access the database

Check your code using techniques such as program analysis (*to be covered in the Systems Security unit*).

Broken Access Control

Lab 3

1. Start up Juice Shop if not already running.
2. Find the Administration page.
3. Once found, use Firefox developer tools, to inspect the files. What interesting data can you extract beyond the Administration page?

Broken Access Control

Violation of the principle of least privilege

Bypassing access control checks by modifying the URL (parameter tampering or force browsing), internal application state, or the HTML page, or by using an attack tool modifying API requests.

Permitting viewing or editing someone else's account, by providing its unique identifier (insecure direct object references)

Accessing API with missing access controls for POST, PUT and DELETE.

Elevation of privilege. Acting as a user without being logged in or acting as an admin when logged in as a user.

Broken Access Control - Mitigations

Deny by default

Implement access control mechanisms once and reuse across application

Enforce record ownership rather than allowing users to create, read, update or delete records

Disable web directory listing and ensure meta-data/backup files, etc. are not accessible/present

Log access control failures and report!

Rate limit API access to mitigate against automated attack tools.

OWASP Top 10 (A01: 2021)

More Input Validation

Lab 4 (Let's try out Burp Suite)

1. Start up Juice Shop if not already running.
2. Go to the Feedback form. Give a zero star rating to the shop.
3. Once you have successfully given a zero star rating, register as an administrator. Note tasks 2 and 3 are not linked. You can complete them in any order.

Cross-site Scripting (XSS)

A type of injection attack

Allows attackers to inject malicious scripts (e. g., JavaScript) into benign websites

Common examples of websites that are vulnerable to XSS attacks are message forums that receive user content and show it to other users

Primary root cause?

Lack of effective input validation mechanisms!

In an XSS attack, the client browser cannot detect the malicious code, since it is sent from the original remote host, i.e. same-origin-policy based security measures are ineffective.

Stored vs. Reflected XSS

Stored (*Also called permanent or Type-I XSS*): The malicious script is permanently stored on the target server (e.g. in a database) and distributed to the victims whenever they request the stored script.

Reflected (*Also called non-permanent or Type-II XSS*): The malicious script is not permanently stored on the target server, but reflected by the server to the victims. A common way of delivering a malicious script is to craft a link to the target website. The link contains the script and clicking the link executes the malicious script in the website's script execution context.

XSS Attacks - Mitigations

What do you think?

Lab 5

1. Start up Juice Shop if not already running.
2. There are two bonus payloads provided in the score board. Copy-paste them into the search box and observe the behaviour.

Error-based SQL Injection

Lab 6

1. Start up Juice Shop if not already running.
2. Extract the database schema and, ideally, the whole database.

You can also try out the sqlmap tool in kali and its documentation. See how you can use it to do this?

Further Reading

<https://owasp.org/Top10/>

<https://cwe.mitre.org>

CyBOK - Web and Mobile Security Knowledge Area (on which this lecture is based): [https://www.cybok.org/media/downloads/](https://www.cybok.org/media/downloads/Web_Mobile_Security_v1.0.1.pdf)
[Web_Mobile_Security_v1.0.1.pdf](https://www.cybok.org/media/downloads/Web_Mobile_Security_v1.0.1.pdf)

More Hands-on Learning

Read the Kali documentation: <https://www.kali.org>

<https://owasp.org/www-project-vulnerableapp/>

<https://owasp.org/www-project-juice-shop/>

<https://tryhackme.com>

<https://www.hackthebox.com>